CHAPTER 15

DESIGN PATTERNS, PRINCIPLES, ARCHITECTURES

111. What design patterns are commonly used in Android apps?

Builder pattern separates construction of a complex object from its representation.

```
AlertDialog.Builder(this) .setTitle("Mahatma Gandhi") .setMessage("Be
the change you wanna see in the world.") .setNegativeButton("No
thanks", { dialogInterface, i -> // "No thanks" button was clicked })
.setPositiveButton("OK", { dialogInterface, i -> // "OK" button was
clicked }) .show()
```

Dependency Injection

```
@Module class AppModule { @Provides fun provideSharedPreferences(app:
Application): SharedPreferences { return
app.getSharedPreferences("prefs", Context.MODE_PRIVATE) } } @Inject
lateinit var sharedPreferences: SharedPreferences
```

Adapter Pattern

```
class MyAdapter(private val data: List) : RecyclerView.Adapter() {
override fun onCreateViewHolder(viewGroup: ViewGroup, i: Int):
MyViewHolder { val inflater = LayoutInflater.from(viewGroup.context)
val view = inflater.inflate(R.layout.row, viewGroup, false) return
MyViewHolder(view) } override fun onBindViewHolder(viewHolder:
MyViewHolder, i: Int) { viewHolder.bind(data[i]) } override fun
getItemCount() = data.size }
```

Facade Pattern

```
interface BooksApi { @GET("books") fun listBooks(): Call> }
```

Command Pattern

```
    class MySpecificEvent { /* Additional fields if needed */ }
    eventBus.register(this) fun onEvent(event: MySpecificEvent) { /* Do
    something */ } eventBus.post(event)
```

Observer Pattern

```
    apiService.getData(someData) .subscribeOn(Schedulers.io())
    .observeOn(AndroidSchedulers.mainThread()) .subscribe { /* an Observer
    */ }
```

MVVM (Model View ViewModel)

The MVVM pattern is trending upwards in popularity.

Important: Design Patterns are crucial for senior developers.